
**Large-Scale Design & Analysis of Neural
Networks
System Test Plan
Version <1.0>
12/5/2024**

Document Control

Distribution List

The following list of people will receive a copy of this document every time a new version of this document becomes available:

Teaching assistants:

Clay Pate

Customer(s):

M. Ilhan Akbas

John M. Thompson

Project team members:

Daniel Machado

Simon Cole

Muhammed Sharif Adepetu

Leah Powell

Change Summary

The following table details changes made between versions of this document:

Version	Date	Modifier	Description
1.1	12/3/2024	Leah	Document Creation, 1.1, 1.2, 1.3, 5.2, 3
1.2	12/4/2024	Muhammed-Sharif	1.4
1.3	12/5/2025	Daniel Machado	2.4, 2.5
1.4	12/5/2024	Leah	4.2, 5.1, 2.4, 2.5, 2.6
1.5	12/5/2024	Muhammed-Sharif	Fleshed out test cases / Formatting
1.6	12/5/2024	Simon Cole	Added/edited test cases

Table of Contents

Document Control	ii
Distribution List.....	ii
Change Summary	ii
1. Introduction	1
1.1. Purpose.....	1
1.2. Scope	1
1.3. System Overview	1
1.4. Testing Approach Overview	1
1.5. Testing Entrance Criteria	2
1.6. Document Overview.....	3
1.7. References.....	3
2. Testing Approach	4
2.1. Testing Types	4
2.1.1. Usability Testing	4
2.1.2. Functionality Testing	4
2.2. Testing Suspension Criteria and Resumption Requirements	4
2.2.1. Suspension Criteria.....	4
2.2.2. Resumption Requirements	4
2.3. Testing Environment	4
2.4. Testing Assumptions	4
2.5. Testing Risks and Contingencies.....	5
2.6. Test Plan.....	6
3. Test Schedule	7
4. Traceability Matrix and Defect Tracking	8
4.1. Traceability Matrix.....	8
4.2. Defect Severity Definitions.....	8
5. Test Cases	9
5.1. Test Case 1	10
5.2. Test Case 2	11
5.3. Test Case 3	12
5.4. Test Case 4	13
5.5. Test Case 5	13

5.6. Test Case 6	14
5.7. Test Case 7	15
5.8 Test Case 8	16
5.9 Test Case 9	17
5.10 Test Case 10	18
5.11 Test Case 11	19
5.14 Test Case 12	20
5.13 Test Case 13	21
5.14 Test Case 14	22
5.15 Test Case 15	23

1. Introduction

1.1. Purpose

The project's goal is to implement a pipeline for large-scale design, creation, and analysis of Bayesian Neural Networks. These networks are using a newer, more modern approach that uses Baye's Theorem for weight distributions. The purpose of this document is to outline a detailed test plan that describes our approach, schedule, and overall organization of testing. The document first covers testing approaches. This includes but is not limited to testing for risks, testing for functionality, and laying out the actual test plan. This document also outlines the test schedule, and actual test cases. Lastly, it covers traceability matrix and defect tracking.

1.2. Scope

The scope of the project includes the release of a github repo for download in order for the user to use the pipeline program.

1. Prototype Release (Version 0.1):

- a. Initial implementation of neural network architecture(s) under study.
- b. Basic data preprocessing pipeline.
- c. Preliminary experiments to validate setup.
- d. Disjointed classes with basic functionality

2. First Analysis Release (Version 0.5):

- a. Results from experiments focusing on the probability distribution of weights for baseline architectures.
- b. Visualizations of weight distributions and their evolution across training epochs.
- c. Comparative study of different activation functions and their impact on weight distributions.

3. Optimization Release (Version 1.0):

- a. Inclusion of optimizations such as alternative training strategies or network modifications (e.g., increasing the number of neurons, changing layer types).
- b. Refined tools for weight analysis.
- c. Comprehensive results and insights into unique patterns in weight distributions.

1.3. System Overview

The system is organized into modules that meet the high-end goals of the project. The modules cover Network Loading, Loss Analysis Tools, Weight Binning & Visualization, Success Criterion Evaluation, and Export/Reporting. The system does not implement any external interfaces as it is a software-based project, organized into individual files that handle all the modules.

1.4. Testing Approach Overview

The testing approach for the neural network analysis project follows a structured, multilevel methodology to ensure robust evaluation of both functionality and performance. This overview links directly to the conceptual diagram outlined in Section 1.3, which depicts the project's components and interactions.

1.4.1.1. Objective and Scope of Testing Levels

1. Unit Testing (Level 1):

- a. **Objective:** Validate the correctness of individual components, such as neural network layers, activation functions, and data preprocessing steps.
- b. **Scope:** Includes testing layer outputs for expected behavior, ensuring accurate implementation of custom functions, and verifying that data preprocessing maintains data integrity.

2. Integration Testing (Level 2):

- a. **Objective:** Ensure seamless interaction between components such as data pipelines, model training modules, and visualization tools.
- b. **Scope:** Covers end-to-end data flow, interaction between training and evaluation phases, and compatibility of third-party libraries.

1.5. Testing Entrance Criteria

To implement system testing, these requirements must be met for testing readiness:

Environment Setup:

- A fully configured testing environment, replicating the production setup as closely as possible, must be available.
- All necessary hardware, software, and network configurations should be in place and verified.
- Access to necessary tools for logging, debugging, and monitoring system performance should be ensured.

Stable Build Availability:

- The system built intended for testing must be stable and deployed to the testing environment.
- All components and modules required for end-to-end testing should be integrated and functional.

Test Data:

- Well-prepared test data sets must be available and validated for use.
- Test data should include edge cases, boundary values, and scenarios for positive and negative testing.

1.6. Document Overview

1.7. References

S. Adepetu, S. Cole, D. Machado, and L. Powell, *Project Proposal Document*. 2024.

S. Adepetu, S. Cole, D. Machado, and L. Powell, *Software Design Documentation*. 2024.

S. Adepetu, S. Cole, D. Machado, and L. Powell, *System Requirements Specification*. 2024

2. Testing Approach

2.1. Testing Types

2.1.1. Usability Testing

Ensure that the required files in the program package are tested for accuracy and general usability. The goal of this testing is to ensure that the files will run without error or unexpected behavior. The user should be able to navigate from file to file without any confusion or issue.

2.1.2. Functionality Testing

Functionality testing will ensure the adherence to the project outlined in the Software Design Document and System Requirement Specifications. The functionality of each module in the system shall be carried out in the runnable program files.

2.2. Testing Suspension Criteria and Resumption Requirements

This section will specify the criteria that will be used to suspend all or a portion of the testing activities on the items associated with this test plan.

2.2.1. Suspension Criteria

Testing will be suspended if the incidents found will not allow further testing of the system/application under-test. If testing is halted, and changes are made to the hardware, software, or database, it is up to the Testing Manager to determine whether the test plan will be re-executed, or part of the plan will be re-executed.

2.2.2. Resumption Requirements

2.3. Testing Environment

The testing environment for this plan is configured to ensure reproducibility, consistency, and compatibility that includes:

- The following test cases will be executed in an environment that runs on Windows.
- The tests will be run in a jupyter notebook, and/or
- The tests will be run on a Docker container

2.4. Testing Assumptions

- There is a pre-built Neural Network Architecture
- There is a processed dataset

- The processed data fits the Neural Network
- Testing environments are compatible
 - The environment software/hardware is sufficient enough to support the necessary components and libraries.
- All libraries and framework versions used are compatible and behave as expected in the environment

2.5. Testing Risks and Contingencies

The risks and contingencies possible for our project are:

1. Data Generation risks – *data is checked, validated, and formatted before it is used*
 - a. improper Data Generation
 - b. issues or Deviations in Input Features
2. Neural Network Generation Risks – *perform preliminary testing with smaller datasets for tuning hyperparameters*
 - a. failure to converge
 - b. dataset noise
3. Excessive Model Failure – *set a reasonable maximum number of attempts*
4. Pipeline Failure / Unexpected Inputs – *include input validation at the appropriate stages of the pipeline and generation, use edge case datasets for testing.*

2.6. Test Plan

Table 1: Test Plan

ID	Test	Status ¹	Date	Notes
1		Open	TBD	N/A
2		Open	TBD	N/A
3		Open	TBD	N/A
4		Open	TBD	N/A
5		Open	TBD	N/A
6		Open	TBD	N/A

¹ Unwritten, Incomplete, Pass, Fail

3. Test Schedule

Testing Schedules: test plans are carried out at every other sprint, and this document is to be managed and written to at least once every sprint keep test plans and test plan results up to date.

4. Traceability Matrix and Defect Tracking

4.1. Traceability Matrix

Table 2: Traceability Matrix

Req. ID	Test Case(s)
UI8, DR1	1
UI1	2
UI2	3
UI3	4
UI4	5
UI5	6

4.2. Defect Severity Definitions

Critical	The defect causes a catastrophic or severe error that results in major problems and the functionality is rendered unavailable to the user. A manual procedure is impossible to be implemented, or high effort is required to remedy the defect. Examples of critical defects are as follows: • System abnormally terminates • Data cannot flow through a business function/lifecycle • Data is corrupted or cannot post to the database
Medium	The defect that does not seriously impair system function can be categorized as a medium defect. A manual procedure requiring medium effort can be implemented to remedy the defect. Examples of medium defects are as follows: • Form navigation is incorrect • Field labels are not consistent with global terminology
Low	The defect is cosmetic or has little to no impact on system functionality. A manual procedure requiring low effort can be implemented to remedy the defect. Examples of low defects are as follows: • Repositioning of fields on screens • Text font on reports is incorrect

5. Test Cases

5.1. Test Case 1

Objective: properly load the .csv file for training

Notes: This test is conducted by initiating the program at which the system shall start the creation inside of GenerateData.py to be saved into a .csv file for training.

Test No.: 1		Current Status: Pending		
Test title: Verify that .csv file is loaded correctly for training				
Testing approach: Black box testing will be used on this test.				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Verify that the environment contains numpy and pandas installed.	Code will not function without these.	The system shall create a fully usable .csv file for training data.	
2	Run the script for GenerateData.py.	This is the file that creates the .csv file.	The code shall run successfully.	
3	Check the correct directory for the "SimpleReg.csv" file.	The file needs to be in the right folder.	The file shall exist in the directory with the name "SimpleReg.csv"	
4	Verify the file contains three columns of (a, b, y).	These are columns used for tensors.	The file shall contain those three columns.	
5	Verify (a, b) contain floating point values and y contains a computed value using the target function.	This is a standard for tensors and matrices in NN training data.	The left columns a and b shall contain floating point values and the corresponding y column shall be correct according to the target function.	
Concluding Remarks: << Filled in by the person who completed the test >>				
Testing Team:		Date Completed:		
<< List members of testing team and lead >>				

5.2. Test Case 2

Objective: *Initialize Class*

Notes: Ensure that the class is properly initialized with all required attributes. Test for proper assignment of values and verify no initialization errors occur

Test No.: 2		Current Status: Pending		
Test title: <i>initialize network</i>				
Testing approach: <i>Instantiate the class, check attribute values, confirm no errors during initialization. Validate that default or input values are correctly assigned</i>				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	<i>Class should accept all the initialization parameters</i>	<i>Instantiate the class that will be used to perform analysis</i>	<i>Class shall be instantiated</i>	
2	<i>Verify attribute values</i>	<i>Check if the attributes match expected values</i>	<i>Attributes are set to the correct values (either default or input).</i>	
3	<i>Check for errors during initialization</i>	<i>Ensure no initialization errors occur</i>	<i>No exceptions or errors are raised during class instantiation.</i>	
Concluding Remarks: << Filled in by the person who completed the test >>				
Testing Team: << List members of testing team and lead >>			Date Completed:	

5.3. Test Case 3

Objective: *Generate Networks initialization*

Notes: *Ensure the network generation process correctly initializes all parameters, including weights, biases, and activation functions. Validate that the network is ready for training without errors.*

Test No.: 3		Current Status: Pending		
Test title: Generating networks				
Testing approach: Generate a network of initializing weights, biases and activation function. Check if network is correctly setup and ready for training				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	After the class has been initialized, the user shall input test data loader, training data loader, number of epochs to train for, loss function, learning rate	All of this is needed to generate and train the networks	All of these parameters shall be accepted by the function	
2	Set up weights and biases	Initialize weights and biases with random or predefined values	Weights and biases are initialized correctly, without errors.	
3	Verify network readiness for training	Check that the network is prepared for training	The network is error-free and ready for training with initialized parameters.	
Concluding Remarks: << Filled in by the person who completed the test >>				
Testing Team: << List members of testing team and lead >>			Date Completed:	

5.4. Test Case 4

Objective: *display single network loss*

Notes: Ensure the loss for a single network is correctly calculated and displayed. Validate the loss calculation and display mechanism

Test No.: 4		Current Status: Pending		
Test title: <i>Displaying the loss graph of a network</i>				
Testing approach: <i>Compute the loss of the network using the loss function, then verify the correct display of the loss value</i>				
STEP	OPERATOR ACTION	PURPOSE	EXEPTED RESULTS	COMMENTS
1	<i>After the class has been initialized, the user shall input the required data to run the training of network and loss graph visualization</i>	<i>All of this is needed to train the network and display the loss graph</i>	<i>The loss graph shall be displayed to the user</i>	
2	Display the calculated loss	Output the loss value to the user interface	The correct loss value is displayed accurately without errors.	
3	Verify the displayed loss	Check the accuracy of the displayed loss value	The loss displayed matches the expected result based on network output.	
Concluding Remarks: << Filled in by the person who completed the test >>				
Testing Team: << List members of testing team and lead >>		Date Completed:		

5.5. Test Case 5

Objective: *Binning Weights Input*

Notes: Ensure that the weights are properly binned according to the predefined binning strategy. Verify that the input weights fall within expected ranges after binning

Test No.: 5		Current Status: Pending		
Test title: Stepping through the different weights				
Testing approach: Test binning of weights by assigning appropriate bins based on predefined intervals and validate their correctness				
STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	After networks have been generated, user shall be able to choose a single layer	The binnings will be created from this layer	The user input shall be accepted by the system and binning of the weights shall start	
2	Apply predefined binning strategy	Organize weights into defined bins	Weights are distributed into the appropriate bins based on value ranges.	
3	Verify weight binning output	Check that binned weights match expected ranges	The binned weights match the predefined ranges without errors.	
Concluding Remarks: << Filled in by the person who completed the test >>				
Testing Team:		Date Completed:		
<< List members of testing team and lead >>				

5.6. Test Case 6

Objective: Binning of weights

Notes: Ensure that the weights are correctly binned according to the specified binning criteria. Verify that each weight is assigned to the appropriate bin and falls within the expected range

Test No.: 6	Current Status: Pending
-------------	-------------------------

<i>Test title: Binning of the weights takes place</i>				
<i>Testing approach: Test the binning process by grouping weights into predefined bins, ensuring correct assignment and verification of weight ranges</i>				
<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	<i>The user shall call the binning weights function</i>	<i>Weights need to be in discrete bins in order to perform statistical analyses</i>	<i>The system shall generate N equal-length bins.</i>	
2	<i>Apply binning criteria and algorithm</i>	Classify weights into bins based on defined ranges	Weights are sorted into bins without errors, adhering to the specified intervals.	
3	Validate binned weights	Ensure all weights are correctly placed in bins	Binned weights are correctly categorized, within expected ranges.	
<i>Concluding Remarks: << Filled in by the person who completed the test >></i>				
<i>Testing Team:</i> <i><< List members of testing team and lead >></i>			<i>Date Completed:</i>	

5.7. Test Case 7

Objective: Accumulation of weight values into bins

Notes: Ensure that weight values are correctly accumulated into their respective bins. Verify that the sum of values in each bin matches expectations based on the predefined binning strategy

<i>Test No.: 7</i>	<i>Current Status: Pending</i>
<i>Test title: Bins are accumulated with weight values</i>	

<i>Testing approach: Test the accumulation process by summing the weight values in each bin and validating the correctness of the accumulated results</i>				
<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	<i>The user shall call the binning weights function</i>	<i>The discrete bins must increment in value for the number of times a weight in that range appeared in the set of networks</i>	<i>The system shall accumulate counts of the weight values into the discrete bins.</i>	
2	Accumulate weight values within bins	Sum the values of weights in each bin	The sum of weight values in each bin is correct according to the defined criteria.	
3	Verify accumulated bin values	Check if the accumulated values match expectations	The accumulated values in each bin align with the expected results.	
<i>Concluding Remarks: << Filled in by the person who completed the test >></i>				
<i>Testing Team:</i> << List members of testing team and lead >>			<i>Date Completed:</i>	

5.8 Test Case 8

Objective: *End network testing once loss has reached a pre-defined limit*

Notes: Verify that the network's loss reaches the pre-defined limit. Terminate further training or testing processes once the condition is satisfied. Ensure the pre-defined limit is correctly configured and assess computational resources based on network size and complexity

<i>Test No.: 8</i>	<i>Current Status: Completed</i>
<i>Test title: End network testing once loss has reached a pre-defined limit</i>	

<i>Testing approach: Monitor real-time loss during testing, verify termination at the threshold and validate against pre-defined criteria for accuracy</i>				
<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
<i>N</i>	Define the loss threshold and configure the network for testing	<i>All of this is needed to generate and train the networks</i>	<i>All these parameters shall be accepted by the function</i>	
<i>Concluding Remarks: << Filled in by the person who completed the test >></i>				
<i>Testing Team:</i> <i><< List members of testing team and lead >></i>			<i>Date Completed:</i>	

5.9 Test Case 9

Objective: *The system only saves NNs which meet a user defined loss-threshold.*

Notes: Ensure the system saves neural networks only if they meet the user-defined loss threshold. Validate threshold configuration and saving functionality

<i>Test No.: 9</i>	<i>Current Status: Pending</i>
<i>Test title: Saving Neural Network's Based on Loss Threshold</i>	
<i>Testing approach: Test Saving functionality by training NNs, comparing their loss to the threshold, and verifying only eligible networks are saved</i>	

STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Train the neural network	Perform training to calculate network loss	The network is trained, and the loss value is computed.	
2	Compare network loss with the threshold	<i>Check if the loss meets the user-defined threshold</i>	Networks with a loss below or equal to the threshold are eligible for saving.	
3	Save networks that meet the threshold	Store the network if it passes the threshold test	Only networks that meet or exceed the loss threshold are saved.	
Concluding Remarks: TBD				
Testing Team: Muhammed-Sharif Adepetu			Date Completed: TBD	

5.10 Test Case 10

Objective: Validate Network Training Convergence

Notes: Ensure that the network converges within the expected number of epochs and does not oscillate. Verify that the training process leads to an acceptable error rate.

Test No.: 10	Current Status: Pending
Test title: Network Convergence Validation	
Testing approach: Train network and monitor error rate to ensure it decreases and converges as expected	

<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	Train the neural network	Start the training process with specified epochs	The network is trained, and the loss value is computed.	
2	Monitor loss accuracy	<i>Track loss & accuracy during training</i>	Loss should steadily decrease, and accuracy should improve.	
3	Verify convergence	Check if the network error stabilizes after several epochs	The training should converge with minimal oscillation in error.	
<i>Concluding Remarks: TBD</i>				
<i>Testing Team:</i> <i>Muhammed-Sharif Adepetu</i>			<i>Date Completed: TBD</i>	

5.11 Test Case 11

Objective: Measure Network Training Time

Notes: Ensure the network training time is within acceptable limits for the given architecture and dataset. Verify that the training process does not exceed the predefined time threshold.

<i>Test No.: 11</i>	<i>Current Status: Pending</i>
<i>Test title: Network Convergence Validation</i>	
<i>Testing approach: Time the network training process and verify it completes within the expected timeframe</i>	

STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Train the neural network	Start the training process with specified epochs	The network is trained, and the loss value is computed.	
2	Measure training time	Track the time taken for the training process	The total training time should not exceed the acceptable threshold.	
3	Compare with expected training time	Verify the training time matches the expected duration	The training time is within the specified limits for the given dataset.	
Concluding Remarks: TBD				
Testing Team:		Date Completed: TBD		

5.14 Test Case 12

Objective: Verify Model Scalability

Notes: Ensure that the model performs well under edge cases, such as highly imbalanced data or noisy inputs. Validate robustness to outliers.

Test No.: 12	Current Status: Pending
Test title: Model Scalability Test	
Testing approach: Test the model with increasing dataset sizes and monitor performance metrics such as training time and accuracy	

STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Run the model on small dataset	Start with small dataset to establish a baseline	The model should train quickly and with expected accuracy	
2	Increase dataset incrementally	Gradually feed larger datasets to model	Training time should increase proportionally without significant slowdowns	
3	Monitor performance	Track training time, accuracy and resource usage	The model should maintain performance even with larger datasets	
Concluding Remarks: TBD				
Testing Team:			Date Completed: TBD	

5.13 Test Case 13

Objective: Validate Model's Ability to Handle Sparse Data

Notes: Ensure that the model performs well when the input data contains significant sparsity

Test No.: 13	Current Status: Pending
Test title: Sparse Data Handling	
Testing approach: Provide sparse datasets to the model and evaluate its ability to learn and make accurate predictions.	

STEP	OPERATOR ACTION	PURPOSE	EXEPECTED RESULTS	COMMENTS
1	Prepare sparse dataset	Create datasets with high percentage of zero values	Sparse datasets should not cause errors during training or evaluation	
2	Train the model on sparce data	Feed sparse data to model during training	The model should be able to process sparse data efficiently without degradation in performance	
3	Evaluate prediction accuracy	Test the model's accuracy on sparse validation data	Accuracy should remain consistent despite sparsity in input data	
Concluding Remarks: TBD				
Testing Team:			Date Completed: TBD	

5.14 Test Case 14

Objective: Verify the functionality of the parameter walkthrough_strategy for traversal control

Notes: Ensure that the system allows for different traversal strategies as specified by the walkthrough_strategy parameter.

Test No.: 14	Current Status: Pending
Test title: Walkthrough Strategy Verification	
Testing approach: Use various walkthrough strategies and verify the correctness of the traversed sequence	

<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	Set walkthrough_strategy to sequential	Test sequential traversal strategy	The system should traverse weights sequentially.	
2	Set walkthrough_strategy to random	Test random traversal strategy	The system should traverse weights in a random order.	
3	Test unsupported strategies	Verify behavior with unsupported strategy	The system should return an error for invalid strategies.	
<i>Concluding Remarks: TBD</i>				
<i>Testing Team:</i>		<i>Date Completed: TBD</i>		

5.15 Test Case 15

Objective: Confirm proper labeling of x-axis using bin_edges and verify their accuracy for the selected layer.

Notes: Ensure the x-axis labels reflect the bin_edges values accurately for the chosen layer.

<i>Test No.: 15</i>	<i>Current Status: Pending</i>
<i>Test title: X-Axis Label Validation</i>	
<i>Testing approach: input various bin_edges configurations and validate the displayed x-axis labels</i>	

<i>STEP</i>	<i>OPERATOR ACTION</i>	<i>PURPOSE</i>	<i>EXEPECTED RESULTS</i>	<i>COMMENTS</i>
1	Provide valid bin_edges input	Test correct x-axis label generation	Labels on the x-axis should match the bin_edges values.	
2	Provide invalid bin_edges input	Test error handling for incorrect bin_edges	The system should return an error for invalid inputs.	
3	Change layer and verify labels	Check x-axis labels for different layers	Labels should update correctly for the selected layer.	
<i>Concluding Remarks: TBD</i>				
<i>Testing Team:</i>			<i>Date Completed: TBD</i>	